

Clases en JavaScript

MANUAL COMPLETO · OBJETOS CON CLASES

De los objetos literales a las clases.

Constructor, métodos, getters, campos privados, herencia, composición y JSON, con 12 ejemplos paso a paso que puedes abrir y probar en tu navegador.

01 Lee el ejemplo del manual

02 Ábrelo en la carpeta ejemplos

03 Cambia el código y observa



JAVASCRIPT · CLASES · LARAGON · NAVEGADOR (F12)

Cada ejemplo tiene su carpeta con index.html, estilos.css y script.js comentado. Complementa a las Prácticas Guiadas de clases.

Alpe formación



Tu ruta de trabajo

ÍNDICE Y REQUISITOS

Este manual explica cómo crear objetos con **clases** en JavaScript. Cada ejemplo está en su propia carpeta de la carpeta **ejemplos**, con HTML, CSS y JavaScript comentados. Léelo aquí, ábrelo en el navegador y cambia el código.



Temas

00 · Cómo usar este manual 3
01 · Del objeto literal a la clase 4
02 · Anatomía de una clase 5
03 · Primera clase 6–7
04 · Literal frente a clase 8–9
05 · Métodos y this 10–11
06 · Getters y setters 12–13
07 · Campos privados (#) 14–15
08 · Miembros estáticos 16–17
09 · Herencia 18–19
10 · Composición 20–21
11 · Colección de objetos 22–23
12 · Clases y JSON 24–25
13 · Clases y DOM: contadores 26–27
14 · Clases y DOM: tareas 28–30
15 · Errores habituales 31
16 · Buenas prácticas 32

Chuleta de sintaxis: pág. 33 · Ejercicios: pág. 34 · Glosario: pág. 35 · Fuentes: pág. 36



Antes de empezar

- [ ] Dominas las variables, las funciones y los arrays de JavaScript.
- [ ] Conoces los objetos literales (propiedades, métodos y `this`).
- [ ] Sabes abrir la consola del navegador con **F12**.
- [ ] Tienes **Laragon** y **Visual Studio Code** instalados.



Cómo usar este manual

LA CARPETA EJEMPLOS Y EL FLUJO DE TRABAJO

El manual y la carpeta **ejemplos** van juntos. En el manual lees cada ejemplo paso a paso; en la carpeta lo abres, lo ejecutas y lo modificas.



Dónde se guardan los ejemplos

Copia la carpeta `ejemplos` dentro de `C:\laragon\www\js\clases\`. Cada ejemplo tiene su propia subcarpeta.





Cómo trabajar con cada ejemplo

1. Inicia Apache con **Iniciar todo** en Laragon.
2. Abre `http://localhost/js/clases/ejemplos/` y pulsa sobre un ejemplo.
3. Abre su carpeta en Visual Studio Code y lee `script.js` mientras pruebas la página.
4. Cambia un valor, guarda con `Ctrl + S` y recarga con `F5`.
5. Abre la consola (`F12`) para ver los mismos resultados y los errores.

Archivo	Qué contiene
`index.html`	La estructura de la página, con comentarios
`estilos.css`	El aspecto, con comentarios. No hace falta tocarlo
`script.js`	El código de la clase, comentado paso a paso

> **Fíjate:** en los ejemplos sin interfaz, la función `mostrar()` escribe cada resultado en la página y en la consola. Es solo una utilidad: el ejemplo está en la clase y en su uso.



01 · Del objeto literal a la clase

TEMA 01 · DE UN OBJETO A UNA PLANTILLA

Un objeto literal es **un objeto concreto**. Una clase es una **plantilla** para crear tantos objetos del mismo tipo como necesites. Cada objeto creado con la plantilla se llama **instancia**.

OBJETO LITERAL

```
const alumno = {
    nombre: "Ana",
    nota: 8,
    aprobado() { return this.nota >= 5; }
};

// Un objeto concreto: para otro
// alumno habría que copiarlo
```

CLASE

```
class Alumno {
    constructor(nombre, nota) {
        this.nombre = nombre;
        this.nota = nota;
    }
    aprobado() { return this.nota >= 5; }
}
const a1 = new Alumno("Ana", 8);
const a2 = new Alumno("Luis", 4);
```

Criterio	Objeto literal	Clase
Cuántos objetos	Uno concreto	Todos los que quieras con la misma estructura
Métodos	Cada literal tiene su propia copia	Se comparten entre todas las instancias
`instanceof`	No pertenece a ninguna clase	Cada objeto pertenece a su clase
Privacidad #	No se admite	Campos y métodos privados
Herencia	No es lo habitual	`extends` y `super`
Úsalo para	Datos puntuales o de configuración	Entidades que se repiten: alumnos, productos, cuentas

Fíjate: las clases de JavaScript son una sintaxis más clara sobre la herencia basada en prototipos que ya existía; no introducen un modelo de herencia nuevo. [1]



02 · Anatomía de una clase

TEMA 02 · LOS ELEMENTOS DE UNA CLASE



Anatomía de una clase

```
class Persona {
    constructor(nombre) { this.nombre = nombre; }
    saludar() { return `Hola, soy ${this.nombre}`; }
}
const ana = new Persona("Ana");
```

Palabras clave class, this, new · **Miembros** constructor y métodos · **Nombre y datos** la clase y sus valores

Elemento	Qué es	Ejemplo
`class`	Palabra clave que declara una clase. Su nombre empieza por mayúscula	`class Persona { ... }`
`constructor`	Método especial que se ejecuta con `new` y prepara el objeto. Solo puede haber uno [2]	`constructor(nombre) { ... }`
Propiedad	Dato del objeto, creado con `this.nombre = ...`	`this.nombre = nombre;`
Método	Función de la clase, sin la palabra `function`	`saludar() { ... }`
Instancia	Objeto creado a partir de la clase	`const ana = new Persona("Ana");`
`new`	Crea la instancia y llama al constructor	`new Persona("Ana")`
`this`	La instancia sobre la que se llama al método	`this.edad++`

Fíjate: las declaraciones de clase **no se elevan**: no puedes usar una clase antes de declararla. Y el cuerpo de una clase se ejecuta siempre en modo estricto. [1]



03 · Mi primera clase

EJEMPLO 01 · CARPETA -PRIMERA-CLASE

Una clase es una plantilla para crear objetos del mismo tipo. Aquí creamos la clase Persona y dos objetos a partir de ella.

Aprenderás a: Declarar una clase con `class` · Escribir el `constructor` y guardar datos con `this` · Crear instancias con `new` y llamar a sus métodos.



Parte 1 · La clase

script.js · parte 1: la clase

```
// PASO 1: declaramos la clase con la palabra "class".
// Por convención, su nombre empieza con mayúscula.
class Persona {

    // PASO 2: el constructor se ejecuta automáticamente al usar "new".
    // Recibe los datos iniciales y los guarda en el objeto con "this".
    constructor(nombre, edad) {
        this.nombre = nombre;   // propiedad "nombre"
        this.edad = edad;       // propiedad "edad"
    }

    // PASO 3: los métodos se escriben sin la palabra "function".
    saludar() {
        return `Hola, soy ${this.nombre} y tengo ${this.edad} años.`;
    }

    cumplirAnios() {
        this.edad++;            // modifica una propiedad del propio objeto
        return `${this.nombre} cumple años: ahora tiene ${this.edad}.`;
    }
}
```



03 · Mi primera clase (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
// PASO 4: creamos objetos (instancias) con "new".
const ana = new Persona("Ana", 22);
const luis = new Persona("Luis", 30);

// PASO 5: usamos los métodos. Cada objeto tiene sus propios datos.
mostrar(ana.saludar());
mostrar(luis.saludar());
mostrar(ana.cumplirAnios());
mostrar(ana.edad);                  // 23: solo ha cambiado Ana
mostrar(luis.edad);                 // 30: Luis no se ha tocado

// PASO 6: comprobamos de qué clase es cada objeto.
mostrar(ana instanceof Persona);    // true
mostrar(typeof Persona);            // function: una clase es una función especial
```

SALIDA EN LA CONSOLA

```
Hola, soy Ana y tengo 22 años.
Hola, soy Luis y tengo 30 años.
Ana cumple años: ahora tiene 23.
23
30
true
function
```



Prueba tú

- Crea un tercer objeto con tu nombre y tu edad y llama a `saludar()`.
- Añade un método `esMayorDeEdad()` que devuelva `true` si la edad es 18 o más.
- Quita la palabra `new` en una de las líneas y lee el error de la consola.

> **Idea clave:** La clase es la plantilla; cada objeto creado con `new` es una instancia con sus propios datos.



04 · Objeto literal frente a clase

EJEMPLO 02 · CARPETA -LITERAL-VS-CLASE

Con objetos literales repetimos la estructura en cada objeto. Con una clase la escribimos una sola vez y todos los objetos la comparten.

Aprenderás a: Ver el problema de repetir objetos literales · Entender que los métodos de una clase se comparten · Usar `instanceof` para saber de qué clase es un objeto.



Parte 1 · La clase

script.js · parte 1: la clase

```
1  // A) CON OBJETOS LITERALES: cada objeto repite sus propiedades y su método.
2  const alumno1 = { nombre: "Ana", nota: 8, aprobado() { return this.nota >= 5; } };
3  const alumno2 = { nombre: "Luis", nota: 4, aprobado() { return this.nota >= 5; } };
4
5  // B) CON UNA CLASE: la estructura se escribe UNA sola vez.
6  class Alumno {
7      constructor(nombre, nota) {
8          this.nombre = nombre;
9          this.nota = nota;
10     }
11
12     aprobado() {
13         return this.nota >= 5;
14     }
15 }
```



04 · Objeto literal frente a clase (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
// Creamos más alumnos, ahora con la clase.
const alumno3 = new Alumno("Marta", 9);
const alumno4 = new Alumno("Pedro", 3);

mostrar(`${alumno1.nombre}: ${alumno1.aprobado()}`);
mostrar(`${alumno3.nombre}: ${alumno3.aprobado()}`);
mostrar(`${alumno4.nombre}: ${alumno4.aprobado()}`);

// Los métodos de una clase se comparten entre todas las instancias...
mostrar(alumno3.aprobado === alumno4.aprobado);    // true
// ...pero cada objeto literal tiene su propia copia de la función.
mostrar(alumno1.aprobado === alumno2.aprobado);    // false

mostrar(alumno3 instanceof Alumno);    // true
mostrar(alumno1 instanceof Alumno);    // false: un literal no pertenece a ninguna clase
mostrar(Object.keys(alumno3).join(", "));   // propiedades propias (el método no aparece)
```

SALIDA EN LA CONSOLA

```
Ana: true
Marta: true
Pedro: false
true
false
true
false
nombre, nota
```



Prueba tú

- Añade un tercer alumno literal y comprueba que hay que copiar otra vez el método.
- Añade a la clase un método `estado()` que devuelva "Aprobado" o "Suspenso".
- Compara dos métodos de la clase con `===` para confirmar que son la misma función.

Idea clave: Con una clase, cada método existe una sola vez y todas las instancias lo comparten.



05 · Métodos, this y encadenamiento

EJEMPLO 03 · CARPETA -METODOS-Y-THIS

Una cuenta bancaria con métodos que modifican sus propias propiedades. Veremos cómo encadenar llamadas y qué ocurre si se pierde this.

Aprenderás a: Usar `this` dentro de los métodos · Devolver `this` para encadenar llamadas · Reconocer el error de perder `this` y arreglarlo con `bind`.



Parte 1 · La clase

script.js · parte 1: la clase

```
1  class CuentaBancaria {
2      // Un parámetro puede tener un valor por defecto: aquí, 0.
3      constructor(titular, saldoInicial = 0) {
4          this.titular = titular;
5          this.saldo = saldoInicial;
6          this.movimientos = [];          // cada cuenta tiene su propio array
7      }
8
9      ingresar(importe) {
10         this.saldo += importe;
11         this.movimientos.push(`+${importe}€`);
12         return this;                    // devolver "this" permite encadenar llamadas
13     }
14
15     retirar(importe) {
16         if (importe > this.saldo) {
17             this.movimientos.push("denegado");
18             return this;
19         }
20         this.saldo -= importe;
21         this.movimientos.push(`-${importe}€`);
22         return this;
23     }
24
25     resumen() {
26         return `${this.titular}: ${this.saldo}€ [${this.movimientos.join(", ")}]`;
27     }
28 }
```



05 · Métodos, this y encadenamiento (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const cuenta = new CuentaBancaria("Lucía", 100);
2
3  // Encadenamos llamadas porque cada método devuelve "this".
4  cuenta.ingresar(50).retirar(30).retirar(500);
5  mostrar(cuenta.resumen());
6
7  // Cada instancia es independiente.
8  const otra = new CuentaBancaria("Pablo");
9  mostrar(otra.resumen());
10
11 // ERROR CLÁSICO: perder "this" al separar el método del objeto.
12 const suelto = cuenta.resumen;
13 try {
14     suelto();
15 } catch (error) {
16     mostrar(error.name);               // TypeError: "this" es undefined
17 }
18
19 // SOLUCIÓN: fijar "this" con bind.
20 const fijado = cuenta.resumen.bind(cuenta);
21 mostrar(fijado());
```

SALIDA EN LA CONSOLA

```
› Lucía: 120€ [+50€, -30€, denegado]
  Pablo: 0€ []
  TypeError
  Lucía: 120€ [+50€, -30€, denegado]
```



Prueba tú

- Añade un método `vaciar()` que ponga el saldo a 0 y devuelva `this`.
- Encadena cinco operaciones seguidas sobre una misma cuenta.
- Sustituye `bind` por una función flecha: `const f = () => cuenta.resumen();`.

> **Idea clave:** `return this` permite encadenar llamadas; `this` se pierde si separas el método del objeto.



06 · Getters y setters con validación

EJEMPLO 04 · CARPETA -GETTERS-Y-SETTERS

Un rectángulo cuyo ancho y alto se validan al asignarlos, con propiedades calculadas (área y perímetro) que solo se pueden leer.

Aprenderás a: Crear getters (`get`) y setters (`set`) · Validar datos al asignarlos · Definir propiedades calculadas de solo lectura.



Parte 1 · La clase

script.js · parte 1: la clase

```
1  class Rectangulo {
2      constructor(ancho, alto) {
3          this.ancho = ancho;     // OJO: esto llama al setter "ancho", no asigna directament
   e
4          this.alto = alto;
5      }
6
7      // GETTER y SETTER de "ancho": guardan el dato real en "_ancho".
8      get ancho() {
9          return this._ancho;
10     }
11     set ancho(valor) {
12         if (valor <= 0) throw new RangeError("El ancho debe ser mayor que 0");
13         this._ancho = valor;
14     }
15
16     get alto() {
17         return this._alto;
18     }
19     set alto(valor) {
20         if (valor <= 0) throw new RangeError("El alto debe ser mayor que 0");
21         this._alto = valor;
22     }
23
24     // GETTERS de solo lectura: propiedades calculadas.
25     get area() {
26         return this.ancho * this.alto;
27     }
28     get perimetro() {
29         return 2 * (this.ancho + this.alto);
30     }
31 }
```



06 · Getters y setters con validación (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const r = new Rectangulo(4, 3);
2  mostrar(r.area);                    // se lee como una propiedad, sin paréntesis
3  mostrar(r.perimetro);
4
5  r.ancho = 10;                       // se asigna como una propiedad: ejecuta el setter
6  mostrar(r.area);
7
8  try { r.alto = -2; } catch (error) { mostrar(`${error.name}: ${error.message}`); }
9  try { new Rectangulo(0, 5); } catch (error) { mostrar(error.message); }
10
11 r.area = 999;                       // "area" no tiene setter: la asignación se ignora
12 mostrar(r.area);
13 mostrar(r._ancho);                  // se puede leer: el guion bajo es solo una convención
```

SALIDA EN LA CONSOLA

```
› 12
  14
  30
  RangeError: El alto debe ser mayor que 0
  El ancho debe ser mayor que 0
  30
  10
```



Prueba tú

- Añade un getter `esCuadrado` que devuelva `true` si ancho y alto son iguales.
- Intenta crear un rectángulo con un ancho negativo y lee el error.
- Añade un setter para asignar ancho y alto a la vez.

Idea clave: Un setter valida al asignar; un getter sin setter es una propiedad de solo lectura.



07 · Campos y métodos privados (#)

EJEMPLO 05 · CARPETA -CAMPOS-PRIVADOS

Una cuenta segura cuyo saldo, historial y PIN no se pueden leer ni modificar desde fuera de la clase.

Aprenderás a: Declarar campos y métodos privados con `#` · Proteger datos y exponerlos solo mediante getters · Devolver copias para no filtrar el estado interno.



Parte 1 · La clase

script.js · parte 1: la clase

```
class CuentaSegura {
    // Los campos privados empiezan por # y se declaran en el cuerpo de la clase.
    #saldo = 0;
    #movimientos = [];
    #pin;

    constructor(titular, pin) {
        this.titular = titular;         // público
        this.#pin = pin;                // privado
    }

    // Un método privado también empieza por #.
    #registrar(texto) {
        this.#movimientos.push(texto);
    }

    ingresar(importe) {
        if (importe <= 0) throw new Error("Importe no válido");
        this.#saldo += importe;
        this.#registrar(`ingreso ${importe}€`);
    }

    retirar(importe, pin) {
        if (pin !== this.#pin) throw new Error("PIN incorrecto");
        if (importe > this.#saldo) throw new Error("Saldo insuficiente");
        this.#saldo -= importe;
        this.#registrar(`retirada ${importe}€`);
    }

    get saldo() { return this.#saldo; }                 // acceso de solo lectura
    get historial() { return [...this.#movimientos]; }  // devolvemos una COPIA
}
```



07 · Campos y métodos privados (#) (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const cuenta = new CuentaSegura("Marta", "1234");
2  cuenta.ingresar(500);
3  cuenta.retirar(120, "1234");
4  mostrar(cuenta.saldo);
5  mostrar(cuenta.historial.join(" | "));
6
7  try { cuenta.retirar(10, "0000"); } catch (e) { mostrar(e.message); }
8  try { cuenta.retirar(9999, "1234"); } catch (e) { mostrar(e.message); }
9
10 cuenta.saldo = 1000000;             // no hay setter: se ignora
11 mostrar(cuenta.saldo);
12
13 cuenta.historial.push("trampa");    // modifica la COPIA, no el historial real
14 mostrar(cuenta.historial.length);
15
16 mostrar(Object.keys(cuenta).join(", "));   // solo se ve la propiedad pública
17 mostrar(JSON.stringify(cuenta));
18
19 // cuenta.#saldo;   // ERROR DE SINTAXIS: ni siquiera se puede escribir fuera de la clase
```

SALIDA EN LA CONSOLA

```
380
ingreso 500€ | retirada 120€
PIN incorrecto
Saldo insuficiente
380
2
titular
{"titular":"Marta"}
```



Prueba tú

- Añade un método `cambiarPin(pinActual, pinNuevo)` que compruebe el PIN actual.
- Escribe `cuenta.#saldo` fuera de la clase y observa el error.
- Añade un getter `ultimoMovimiento`.

> **Idea clave:** Lo privado solo existe dentro de la clase: expón únicamente lo necesario. [3]



08 · Miembros estáticos (static)

EJEMPLO 06 · CARPETA -MIEMBROS-ESTATICOS

Un contador de productos y una utilidad de ordenación que pertenecen a la clase, no a cada objeto.

Aprenderás a: Declarar campos y métodos con `static` · Llamar a un estático con el nombre de la clase · Usar un contador compartido por todas las instancias.



Parte 1 · La clase

script.js · parte 1: la clase

```
1  class Producto {
2      static #contador = 0;       // campo estático PRIVADO: pertenece a la clase
3      static IVA = 0.21;          // campo estático público
4
5      constructor(nombre, precio) {
6          Producto.#contador++;               // se accede con el nombre de la clase
7          this.id = Producto.#contador;       // cada producto recibe un id distinto
8          this.nombre = nombre;
9          this.precio = precio;
10     }
11
12     // Método de INSTANCIA: se llama sobre un objeto.
13     precioConIva() {
14         return Number((this.precio * (1 + Producto.IVA)).toFixed(2));
15     }
16
17     // Métodos ESTÁTICOS: se llaman sobre la clase, no sobre un objeto.
18     static total() {
19         return Producto.#contador;
20     }
21
22     static comparar(a, b) {                 // sirve para ordenar por precio
23         return a.precio - b.precio;
24     }
25 }
```



08 · Miembros estáticos (static) (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const teclado = new Producto("Teclado", 59.90);
2  const raton = new Producto("Ratón", 25);
3  const monitor = new Producto("Monitor", 140);
4
5  mostrar(`${teclado.id}, ${raton.id}, ${monitor.id}`);
6  mostrar(Producto.total());                  // se llama sobre la clase
7  mostrar(teclado.precioConIva());
8  mostrar(Producto.IVA);
9
10 const ordenados = [teclado, raton, monitor].sort(Producto.comparar);
11 mostrar(ordenados.map(p => p.nombre).join(" < "));
12
13 mostrar(typeof teclado.total);              // undefined: los estáticos no están en las ins
   tancias
```

SALIDA EN LA CONSOLA

```
› 1, 2, 3
  3
  72.48
  0.21
  Ratón < Teclado < Monitor
  undefined
```



Prueba tú

- Crea otros dos productos y comprueba sus ids.
- Cambia `Producto.IVA` a `0.10` y vuelve a calcular.
- Añade un método estático `masBarato(lista)`.

> **Idea clave:** Los miembros estáticos pertenecen a la clase, no a cada instancia.



09 · Herencia: extends y super

EJEMPLO 07 · CARPETA -HERENCIA

La clase Animal define lo común. Perro y Gato heredan de ella, añaden lo suyo y sobrescriben métodos.

Aprenderás a: Crear subclases con `extends` · Llamar al constructor del padre con `super()` · Sobrescribir métodos y entender el polimorfismo.



Parte 1 · La clase

script.js · parte 1: la clase

```
class Animal {
    constructor(nombre) {
        this.nombre = nombre;
    }
    hablar() {
        return `${this.nombre} hace un sonido.`;
    }
    presentar() {
        return `Soy ${this.nombre}.`;
    }
}

// "extends": Perro HEREDA todo lo de Animal.
class Perro extends Animal {
    constructor(nombre, raza) {
        super(nombre);          // llama al constructor del padre (siempre lo primero)
        this.raza = raza;       // propiedad nueva, solo de Perro
    }
    hablar() {                  // SOBRESCRIBE el método del padre
        return `${this.nombre} ladra: ¡Guau!`;
    }
}

class Gato extends Animal {
    hablar() {                  // reutiliza el método del padre con "super"
        return `${super.hablar()} Y además maúlla.`;
    }
}
```



09 · Herencia: extends y super (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const animales = [new Perro("Rex", "Labrador"), new Gato("Misi"), new Animal("Bicho")];
2
3  // POLIMORFISMO: la misma llamada, un comportamiento distinto en cada clase.
4  for (const animal of animales) {
5      mostrar(animal.hablar());
6  }
7
8  mostrar(animales[0].presentar());          // método heredado: no está escrito en Perro
9  mostrar(animales[0].raza);
10 mostrar(animales[1].raza);                 // undefined: Gato no tiene raza
11
12 mostrar(animales[0] instanceof Perro);     // true
13 mostrar(animales[0] instanceof Animal);    // true: un Perro ES un Animal
14 mostrar(animales[1] instanceof Perro);     // false
```

SALIDA EN LA CONSOLA

```
› Rex ladra: ¡Guau!
  Misi hace un sonido. Y además maúlla.
  Bicho hace un sonido.
  Soy Rex.
  Labrador
  undefined
  true
  true
  false
```



Prueba tú

- Crea la clase `Vaca` que herede de `Animal` y sobrescriba `hablar()`.
- Añade a `Perro` un método `moverCola()`.
- Comprueba con `instanceof` de qué clases es un `Gato`.

> **Idea clave:** Una subclase hereda todo, añade lo suyo y puede sobrescribir métodos. `super()` va siempre primero. [4]



10 · Composición: objetos dentro de objetos

EJEMPLO 08 · CARPETA -COMPOSICION

Un carrito formado por líneas, y cada línea por un producto. Es la alternativa a la herencia cuando un objeto "tiene" otros.

Aprenderás a: Diseñar clases que se componen de otras clases · Distinguir "es un" (herencia) de "tiene un" (composición) · Usar un array privado para guardar objetos.



Parte 1 · La clase

script.js · parte 1: la clase

```
class Producto {
    constructor(nombre, precio) {
        this.nombre = nombre;
        this.precio = precio;
    }
}

// Una línea del carrito TIENE un producto y una cantidad.
class LineaCarrito {
    constructor(producto, cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
    }
    get subtotal() {
        return this.producto.precio * this.cantidad;
    }
}

// El carrito TIENE una lista de líneas (privada).
class Carrito {
    #lineas = [];

    agregar(producto, cantidad = 1) {
        const existente = this.#lineas.find(l => l.producto.nombre === producto.nombre);
        if (existente) {
            existente.cantidad += cantidad;     // ya estaba: sumamos unidades
        } else {
            this.#lineas.push(new LineaCarrito(producto, cantidad));
        }
        return this;
    }

    get total() {
        return this.#lineas.reduce((suma, linea) => suma + linea.subtotal, 0);
    }

    resumen() {
        return this.#lineas.map(l => `${l.cantidad} x ${l.producto.nombre} = ${l.subtotal.t
oFixed(2)}€`);
    }
}
```



10 · Composición: objetos dentro de objetos (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const teclado = new Producto("Teclado", 59.90);
2  const raton = new Producto("Ratón", 25);
3
4  const carrito = new Carrito();
5  carrito.agregar(teclado, 2).agregar(raton).agregar(teclado);
6
7  for (const linea of carrito.resumen()) {
8      mostrar(linea);
9  }
10 mostrar(`Total: ${carrito.total.toFixed(2)}€`);
```

SALIDA EN LA CONSOLA

```
› 3 x Teclado = 179.70€
  1 x Ratón = 25.00€
  Total: 204.70€
```



Prueba tú

- Añade un método `quitar(nombre)` al carrito.
- Añade un getter `unidades` con el total de unidades.
- Aplica un 10 % de descuento si el total supera 100 €.

> **Idea clave:** Composición es «tiene un»; herencia es «es un». Elige la que describa la relación real.



11 · Una colección de objetos: la biblioteca

EJEMPLO 09 · CARPETA -COLECCION-DE-OBJETOS

La clase Biblioteca gestiona una lista de objetos Libro: añadir, buscar, prestar y devolver.

Aprenderás a: Gestionar un array de objetos dentro de una clase · Usar `find` y `filter` sobre instancias · Personalizar la conversión a texto con `toString()`.



Parte 1 · La clase

script.js · parte 1: la clase

```
class Libro {
    constructor(titulo, autor, anio) {
        this.titulo = titulo;
        this.autor = autor;
        this.anio = anio;
        this.prestado = false;
    }
    toString() {                                // se usa al convertir el objeto a texto
        return `${this.titulo} (${this.autor}, ${this.anio})${this.prestado ? " [prestado]"
: ""}`;
    }
}

class Biblioteca {
    #libros = [];

    agregar(libro) {
        this.#libros.push(libro);
        return this;
    }

    buscar(texto) {                             // filtra por título o autor
        const t = texto.toLowerCase();
        return this.#libros.filter(l => l.titulo.toLowerCase().includes(t) || l.autor.toLow
erCase().includes(t));
    }

    prestar(titulo) {
        const libro = this.#libros.find(l => l.titulo === titulo);
        if (!libro) return "No existe ese libro";
        if (libro.prestado) return "Ya está prestado";
        libro.prestado = true;
        return `Prestado: ${libro.titulo}`;
    }

    devolver(titulo) {
        const libro = this.#libros.find(l => l.titulo === titulo);
        if (libro) libro.prestado = false;
    }

    get disponibles() {
        return this.#libros.filter(l => !l.prestado);
    }
}
```



11 · Una colección de objetos: la biblioteca (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const biblioteca = new Biblioteca();
2  biblioteca
3      .agregar(new Libro("El Quijote", "Cervantes", 1605))
4      .agregar(new Libro("La Celestina", "Fernando de Rojas", 1499))
5      .agregar(new Libro("Rimas", "Bécquer", 1871));
6
7  mostrar(biblioteca.prestar("El Quijote"));
8  mostrar(biblioteca.prestar("El Quijote"));
9  mostrar(biblioteca.prestar("Don Juan"));
10
11 mostrar(`Disponibles: ${biblioteca.disponibles.length}`);
12 mostrar(biblioteca.buscar("cervantes").join(" | "));   // join llama a toString()
13
14 biblioteca.devolver("El Quijote");
15 mostrar(`Disponibles: ${biblioteca.disponibles.length}`);
```

SALIDA EN LA CONSOLA

```
› Prestado: El Quijote
  Ya está prestado
  No existe ese libro
  Disponibles: 2
  El Quijote (Cervantes, 1605) [prestado]
  Disponibles: 3
```



Prueba tú

- Añade un libro más y busca por una parte del título.
- Añade un getter `prestados`.
- Impide agregar dos libros con el mismo título.

Idea clave: La clase colección encapsula el array y expone solo las operaciones con sentido.



12 · Clases y JSON: guardar y recuperar

EJEMPLO 10 · CARPETA -CLASES-Y-JSON

JSON solo guarda datos, no métodos. Veremos cómo controlar qué se guarda con toJSON() y cómo reconstruir instancias con un método estático.

Aprenderás a: Entender qué pierde un objeto al pasar por JSON · Controlar la serialización con `toJSON()` · Reconstruir instancias con un método estático.



Parte 1 · La clase

script.js · parte 1: la clase

```
class Tarea {
    constructor(texto, hecha = false, id = Date.now()) {
        this.texto = texto;
        this.hecha = hecha;
        this.id = id;
    }

    alternar() {
        this.hecha = !this.hecha;
        return this;
    }

    // JSON.stringify llama a toJSON(): decidimos qué datos se guardan.
    toJSON() {
        return { id: this.id, texto: this.texto, hecha: this.hecha };
    }

    // Método estático que RECONSTRUYE una instancia a partir de datos planos.
    static desdeJSON(datos) {
        return new Tarea(datos.texto, datos.hecha, datos.id);
    }
}
```



12 · Clases y JSON: guardar y recuperar (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  const t1 = new Tarea("Estudiar clases", false, 1);
2  const t2 = new Tarea("Hacer la práctica", true, 2);
3
4  const texto = JSON.stringify([t1, t2]);          // instancias -> texto
5  mostrar(texto);
6
7  const planos = JSON.parse(texto);                // texto -> objetos PLANOS
8  mostrar(typeof planos[0].alternar);              // undefined: se han perdido los métodos
9  mostrar(planos[0] instanceof Tarea);             // false
10
11 const tareas = planos.map(datos => Tarea.desdeJSON(datos));   // reconstruimos
12 mostrar(tareas[0] instanceof Tarea);             // true
13 mostrar(tareas[0].alternar().hecha);             // true: los métodos funcionan otra vez
14
15 localStorage.setItem("tareas-ejemplo", texto);  // lo guardamos en el navegador
16 mostrar(localStorage.getItem("tareas-ejemplo") === texto);
```

SALIDA EN LA CONSOLA

```
› [{"id":1,"texto":"Estudiar clases","hecha":false},{"id":2,"texto":"Hacer la práctica","hech
  a":true}]
  undefined
  false
  true
  true
  true
```



Prueba tú

- Añade la propiedad `prioridad` a `Tarea` y comprueba que se guarda.
- Quita `toJSON()` y observa el texto que se genera.
- Llama a `localStorage.getItem` sin haber guardado nada.

> **Idea clave:** JSON guarda datos, no métodos: hay que reconstruir las instancias al recuperarlas. [5] [6]



13 · Clases y DOM: contadores independientes

EJEMPLO 11 · CARPETA -CLASE-Y-DOM-CONTADOR

Una clase Contador controla un trozo de la página. Creamos dos contadores y cada uno funciona por su cuenta.

Aprenderás a: Asociar una clase a un elemento del DOM · Conservar `this` en los eventos con funciones flecha · Crear varias instancias independientes.



Parte 1 · La clase

script.js · parte 1: la clase

```
class Contador {
    #valor;
    #inicio;

    // Recibe el elemento HTML que debe controlar.
    constructor(elemento, inicio = 0, paso = 1) {
        this.elemento = elemento;
        this.pantalla = elemento.querySelector(".valor");
        this.paso = paso;
        this.#inicio = inicio;
        this.#valor = inicio;

        // Usamos funciones FLECHA: así "this" sigue siendo el Contador dentro del evento.
        elemento.querySelector(".mas").addEventListener("click", () => this.sumar());
        elemento.querySelector(".menos").addEventListener("click", () => this.restar());
        elemento.querySelector(".reiniciar").addEventListener("click", () => this.reiniciar
());

        this.dibujar();
    }

    sumar() {
        this.#valor += this.paso;
        this.dibujar();
    }

    restar() {
        this.#valor -= this.paso;
        this.dibujar();
    }

    reiniciar() {
        this.#valor = this.#inicio;
        this.dibujar();
    }

    dibujar() {
        this.pantalla.textContent = this.#valor;
    }
}
```



13 · Clases y DOM: contadores independientes (continuación)

PARTE 2 DE 2



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
1  // Cada instancia controla SU propio trozo de HTML y guarda SU propio valor.
2  const contadorA = new Contador(document.getElementById("contadorA"));
3  const contadorB = new Contador(document.getElementById("contadorB"), 10, 5);
```



Así se ve en el navegador

Ejemplo 11 · Clases y DOM: contadores independientes

Una clase Contador controla un trozo de la página. Creamos dos contadores y cada uno funciona por su cuenta.

Contador A

2

− + Reiniciar

Contador B (de 5 en 5)

15

− + Reiniciar

Abre `script.js` para leer el código comentado.



Prueba tú

- Crea un tercer contador que cuente de 2 en 2 desde 100.
- Cambia las funciones flecha por funciones normales y lee el error.
- Añade un límite máximo de 20.

Idea clave: Cada instancia controla su propio trozo de HTML y guarda su propio estado.



14 · Clases y DOM: lista de tareas con localStorage

EJEMPLO 12 · CARPETA -CLASES-Y-DOM-TAREAS

Una aplicación completa: la clase Tarea representa un dato, la clase GestorTareas guarda y modifica la lista, y el resto del código conecta con el HTML.

Aprenderás a: Separar los datos (Tarea) de la gestión (GestorTareas) · Guardar y cargar la lista con JSON y localStorage · Conectar clases con formularios y listas del DOM.



Parte 1 · La clase

script.js · parte 1: la clase

```
class Tarea {
    constructor(texto, hecha = false, id = Date.now()) {
        this.texto = texto;
        this.hecha = hecha;
        this.id = id;
    }
}

class GestorTareas {
    #tareas = [];
    #clave;

    constructor(clave) {
        this.#clave = clave;
        this.#cargar();                          // al crear el gestor, recupera lo guardado
    }

    agregar(texto) {
        this.#tareas.push(new Tarea(texto));
        this.#guardar();
    }

    alternar(id) {
        const tarea = this.#tareas.find(t => t.id === id);
        if (tarea) tarea.hecha = !tarea.hecha;
        this.#guardar();
    }

    eliminar(id) {
        this.#tareas = this.#tareas.filter(t => t.id !== id);
        this.#guardar();
    }

    get tareas() { return [...this.#tareas]; }
    get pendientes() { return this.#tareas.filter(t => !t.hecha).length; }

    #guardar() {
        localStorage.setItem(this.#clave, JSON.stringify(this.#tareas));
    }

    #cargar() {
        const datos = JSON.parse(localStorage.getItem(this.#clave)) || [];
        this.#tareas = datos.map(d => new Tarea(d.texto, d.hecha, d.id));
    }
}
```



14 · Clases y DOM: lista de tareas con localStorage (continuación)

PARTE 2 DE 3



Parte 2 · Usar la clase

script.js · parte 2: uso de la clase

```
// Conectamos el gestor con el HTML.
const gestor = new GestorTareas("tareas-ejemplo-12");
const formulario = document.getElementById("formulario");
const campo = document.getElementById("campoTexto");
const lista = document.getElementById("lista");
const resumen = document.getElementById("resumen");

function dibujar() {
    lista.textContent = "";
    for (const tarea of gestor.tareas) {
        const li = document.createElement("li");
        li.className = tarea.hecha ? "hecha" : "";

        const texto = document.createElement("span");
        texto.textContent = tarea.texto;
        texto.addEventListener("click", () => { gestor.alternar(tarea.id); dibujar(); });

        const borrar = document.createElement("button");
        borrar.textContent = "Borrar";
        borrar.addEventListener("click", () => { gestor.eliminar(tarea.id); dibujar(); });

        li.append(texto, borrar);
        lista.appendChild(li);
    }
    resumen.textContent = `${gestor.pendientes} pendientes de ${gestor.tareas.length}`;
}

formulario.addEventListener("submit", (evento) => {
    evento.preventDefault();                    // evita que la página se recargue
    const texto = campo.value.trim();
    if (texto === "") return;
    gestor.agregar(texto);
    campo.value = "";
    dibujar();
});

dibujar();
```



14 · Clases y DOM: lista de tareas con localStorage (continuación)

PARTE 3 DE 3



Así se ve en el navegador





Prueba tú

- Añade la propiedad `fecha` a `Tarea`.
- Añade un botón «Borrar hechas».
- Abre `F12` → **Aplicación** y mira el JSON guardado.

> **Idea clave:** Separa los datos (`Tarea`), la lógica (`GestorTareas`) y la interfaz (el DOM).



15 · Errores habituales

TEMA 15 · ANTES DE PEDIR AYUDA

Estos son los fallos más frecuentes al empezar con clases. Los mensajes son los de Chrome y Edge; en Firefox cambia el texto, no la causa.

Error	Causa	Forma correcta
`TypeError: Class constructor P cannot be invoked without 'new'`	Se ha llamado a la clase como una función	`new Persona("Ana")`
`ReferenceError: Must call super constructor in derived class before accessing 'this'...`	En una subclase se usa `this` antes de `super()`	`super(...)` siempre en la primera línea del constructor
`ReferenceError: Cannot access 'Q' before initialization`	Se usa la clase antes de declararla	Declara la clase antes de crear objetos
`TypeError: Cannot read properties of undefined (reading 'n')`	Se perdió `this` al separar el método	Llamar `objeto.metodo()` o usar `bind` o una flecha
`SyntaxError: A class may only have one constructor`	Hay dos constructores en la clase	Un único `constructor`
`RangeError: Maximum call stack size exceeded`	Un setter se asigna a sí mismo: `this.ancho = v`	Guardar en otro campo: `this._ancho = v`
`TypeError: ... .t is not a function`	Se llama a un método estático sobre una instancia	`Clase.t()`, no `objeto.t()`
`SyntaxError: Unexpected token ','`	Se han puesto comas entre los métodos de la clase	Los miembros de una clase no llevan comas
`TypeError: o.m is not a function` tras `JSON.parse`	JSON devuelve objetos planos sin métodos	Reconstruir con un método estático `desdeJSON`
`TypeError: Cannot read private member #p from an object whose class did not declare it`	Se lee un campo privado de un objeto de otra clase	Acceder a `#p` solo con objetos de esa clase



16 · Buenas prácticas

TEMA 16 · DISEÑAR BUENAS CLASES

Una clase bien diseñada es pequeña, tiene un propósito claro y protege sus datos. Estas pautas te ayudarán a diseñarlas.



Buenas prácticas

- **Nombres claros:** las clases en singular y con mayúscula (`Producto`), los métodos con verbos (`calcularTotal`).
- **Una responsabilidad:** `Tarea` guarda los datos de una tarea; `GestorTareas` gestiona la lista. No lo mezcles todo.
- **Privado por defecto:** declara con `#` todo lo que no deba tocarse desde fuera y expón solo lo necesario.
- **Devuelve copias:** un getter que devuelve el array interno permite modificarlo desde fuera. Devuelve `[...this.#lista]`.
- **Valida en los setters y constructores:** impide que el objeto llegue a un estado incoherente.
- **Métodos cortos:** si un método hace tres cosas, sepáralo en tres.



¿Clase, objeto literal o herencia?

Situación	Usa
Un dato puntual: configuración, opciones, un único objeto	Objeto literal
Muchos objetos con la misma estructura y comportamiento	Clase
Un tipo es una versión especializada de otro («es un»)	Herencia con `extends`
Un objeto está formado por otros («tiene un»)	Composición
Datos compartidos o utilidades que no dependen de una instancia	Miembros `static`

> **Fíjate:** si dudas entre herencia y composición, empieza por la composición. Es más flexible y evita jerarquías profundas.



Chuleta de sintaxis

REFERENCIA RÁPIDA · CLASES

Resumen de la sintaxis de clases que has visto en este manual.

Propiedad	Para qué sirve	Ejemplo
`class A { }`	Declara una clase	`class Persona { ... }`
`constructor()`	Prepara cada objeto nuevo	`constructor(n) { this.n = n; }`
`new A()`	Crea una instancia	`const p = new Persona("Ana");`
`metodo() { }`	Método de instancia	`saludar() { return this.n; }`
`this`	La instancia actual	`this.edad++;`
`return this`	Permite encadenar llamadas	`c.ingresar(5).retirar(2)`
`get x() { }`	Propiedad calculada de lectura	`get area() { ... }`
`set x(v) { }`	Asignación con validación	`set ancho(v) { ... }`
`#campo`	Campo privado	`#saldo = 0;`
`#metodo() { }`	Método privado	`#registrar(t) { ... }`
`static campo`	Dato de la clase	`static IVA = 0.21;`
`static metodo()`	Método de la clase	`static total() { ... }`
`extends`	Hereda de otra clase	`class Perro extends Animal`
`super(...)`	Llama al constructor del padre	`super(nombre);`
`super.metodo()`	Llama al método del padre	`super.hablar()`
`instanceof`	Comprueba la clase de un objeto	`p instanceof Persona`
`toString()`	Conversión a texto	`${libro}`
`toJSON()`	Qué datos guarda `JSON.stringify`	`toJSON() { return {...}; }`
`static desdeJSON(d)`	Reconstruye una instancia	`Tarea.desdeJSON(datos)`
`throw new Error()`	Lanza un error controlado	`throw new Error("PIN incorrecto");`
`try { } catch (e) { }`	Captura el error	`catch (e) { e.message }`
`obj.metodo.bind(obj)`	Fija `this`	`const f = c.resumen.bind(c);`



Practica lo aprendido

EJERCICIOS PROPUESTOS

Crea una carpeta por ejercicio con `index.html`, `estilos.css` y `script.js`. Puedes partir de cualquier ejemplo de la carpeta **ejemplos**.



Ejercicios propuestos

- [ ] **01 · Mascota.** Clase `Mascota` con nombre y edad, y un método `presentarse()`.
- [ ] **02 · Triángulo.** Clase `Triangulo` con tres lados y un getter `perimetro`.
- [ ] **03 · Termómetro.** Setter que rechace temperaturas por debajo de −273,15 y getter en Fahrenheit.
- [ ] **04 · Contraseña.** Clase `Usuario` con `#clave` privada y un método `comprobar(clave)`.
- [ ] **05 · Entradas.** Clase `Entrada` con un contador estático que numere cada entrada vendida.
- [ ] **06 · Vehículos.** `Vehiculo` y dos subclases con un método `describir()` sobrescrito.
- [ ] **07 · Pedido.** Un `Pedido` formado por varias `LineaPedido`, con total y número de unidades.
- [ ] **08 · Videoclub.** Clase `Pelicula` y clase `Videoclub` con alquilar y devolver.
- [ ] **09 · Notas.** Clase `Nota` guardada en `localStorage` con `toJSON()` y `desdeJSON()`.
- [ ] **10 · Cronómetro.** Clase `Cronometro` que controle un elemento de la página, como los contadores.
- [ ] **11 · Agenda.** Clase `Contacto` y clase `Agenda` con buscador en pantalla.
- [ ] **12 · Mini tienda.** Productos físicos y digitales con un carrito, como en la práctica 6.

Siguiente paso: aplica todo esto en las **Prácticas Guiadas de clases**: seis prácticas con el HTML y el CSS ya hechos, en las que solo escribes el JavaScript.



Glosario

TÉRMINOS CLAVE

Término	Definición
Clase	Plantilla para crear objetos del mismo tipo.
Instancia	Objeto creado a partir de una clase con `new`.
Constructor	Método especial que inicializa cada instancia. Solo hay uno por clase.
Propiedad	Dato guardado en una instancia o en la clase.
Método	Función definida dentro de una clase.
`this`	La instancia sobre la que se ejecuta el método.
Getter / setter	Métodos que se usan como propiedades: leer y asignar con lógica.
Campo privado	Miembro con `#` accesible solo dentro de la clase.
Miembro estático	Miembro de la clase, no de las instancias (`static`).
Herencia	Una clase hija obtiene los miembros de una clase padre (`extends`).
Subclase / superclase	Clase hija y clase padre de una relación de herencia.
`super`	Acceso al constructor y a los métodos de la clase padre.
Sobrescribir	Redefinir en la subclase un método heredado.
Polimorfismo	La misma llamada se comporta de forma distinta según la clase del objeto.
Composición	Un objeto formado por otros objetos («tiene un»).
Encadenamiento	Llamar varios métodos seguidos porque cada uno devuelve `this`.
Serializar	Convertir una instancia en texto (`JSON.stringify`).



Fuentes y Documentación Oficial

REFERENCIAS DEL MANUAL · CONSULTA: 13/11/2026

Cada número entre corchetes remite a la documentación correspondiente. Los ejemplos de este manual se han ejecutado y comprobado en Node.js y en el navegador.



Fuentes y Documentación Oficial

[1] Clases · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Classes

[2] constructor · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes/constructor

[3] Private elements · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes/Private_elements

[4] extends · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Classes/extends

[5] JSON.stringify() · MDN Web Docs · developer.mozilla.org/JavaScript/Reference/Global_Objects/JSON/stringify

[6] Window.localStorage · MDN Web Docs · developer.mozilla.org/es/docs/Web/API/Window/localStorage

> **Dónde seguir:** para profundizar, consulta en MDN los campos públicos de clase, los métodos de acceso y la herencia de objetos integrados.